SiCaS

Yellow Paper

Formal Specification and Verification Obligations

SiCaS Contributors

Version 1.0.0 31 October 2026

Document	Yellow paper
Version	1.0.0
Date	31 October 2026
Code commit	`ef13675`
Release build commit	`123eb69`
Evidence tag	`final-audit-2026-10-31-dpa`
Public label	SAL-2 locally
Production status	Not certified, not audited, not hardware-validated
License	GPL-3.0-only

Abstract

This yellow paper specifies the functional behaviour, data formats, security definitions, and verification obligations of SiCaS, a safe-Rust workspace for first-order side-channel attenuation and leakage assessment. It defines the two-share Boolean masking model, the masked operations, the masked ChaCha20 reference integration, the 64-block bitsliced batch path, the trace format, the TVLA/CPA/DPA algorithms, the timing and leakage-model checks, and the release manifest. It states the correctness theorems and the first-order security argument, lists the assumptions under which those statements hold, and maps each public claim to a proof obligation and an evidence artifact. The specification is deliberately explicit about what is not proven: physical leakage, higher-order attacks, fault injection, template attacks, production zeroization, and independent audit remain outside the current claim set. The document is a specification and verification plan, not a substitute for independent review.

Keywords: formal specification, Boolean masking, TVLA, CPA, DPA, ChaCha20, Rust, proof obligations, assurance.

Contents

1 Scope and relationship to the white paper **4**

2 Notation and conventions **4**

3 Formal model **5**
3.1 Values, shares, and reconstruction . . . 5
3.2 Leakage functions and first-order game . . . 5
3.3 Assumptions . . . 6

4 Core data types and invariants **6**
4.1 `Choice` . . . 6
4.2 `Secret<T>` . . . 6
4.3 `Masked<T>` . . . 6
4.4 `EntropySource` . . . 7
4.5 Invariants . . . 7

5 Constant-time primitives **7**
5.1 Comparison . . . 7
5.2 Selection and swap . . . 7
5.3 Table lookup . . . 7

6 Masked arithmetic **8**
6.1 XOR, complement, shifts, and rotations . . . 8
6.2 Refresh . . . 8
6.3 AND . . . 8
6.4 Masked addition . . . 9
6.5 Selection, equality, and bit extraction . . . 9

7 ChaCha20 specification **9**
7.1 State and constants . . . 9
7.2 Quarter round . . . 10
7.3 Block function . . . 10
7.4 Masked scalar implementation . . . 10
7.5 Bitsliced batch implementation . . . 10
7.6 Transpose . . . 11

8 Harness specification **11**
8.1 Trace format . . . 11
8.2 Preprocessing . . . 12
8.3 TVLA . . . 12
8.4 CPA . . . 12
8.5 DPA . . . 13
8.6 Timing and leakage model . . . 13

9 Security definitions and claims **13**
9.1 First-order security definition . . . 13
9.2 Claims . . . 13
9.3 Proof obligations . . . 14

10 Verification methodology **15**
10.1 Testing . . . 15
10.2 Kani . . . 15
10.3 Miri and sanitizers . . . 15
10.4 Fuzzing and mutation . . . 15
10.5 Assembly and timing . . . 15
10.6 Release verification . . . 15

11 Data formats **15**
11.1 Trace and plaintext files . . . 15
11.2 Reports . . . 16
11.3 Release manifest . . . 16
11.4 Evidence bundle . . . 16

12 Error model and fail-closed behaviour **16**

13 Performance model **17**
13.1 Complexity . . . 17
13.2 Measured results . . . 17

14 Conformance and test vectors **17**
14.1 RFC 8439 . . . 17
14.2 Differential statistics . . . 17
14.3 Release conformance . . . 17

15 Versioning and change log **17**

A API index **18**

B Conformance checklist **18**

1 Scope and relationship to the white paper

This document is the formal companion to the SiCaS white paper. The white paper explains the motivation, architecture, and evaluation at a technical-management level. The yellow paper specifies:

1. the data types and invariants of the core masking library;
2. the functional behaviour of the masked operations;
3. the ChaCha20 reference integration and the bitsliced batch path;
4. the trace format and the TVLA, CPA, and DPA algorithms;
5. the security definitions and the assumptions behind the first-order claim;
6. the verification obligations and the evidence mapping;
7. the release, provenance, and reproducibility formats.

The specification is written against code commit `ef13675` and evidence tag `final-audit-2026-10-31-dpa`. It is a *specification of the current implementation*, not a specification of an ideal cryptographic library. Where the implementation has a known limitation, the limitation is stated.

2 Notation and conventions

$\{0,1\}$ The set $\{0,1\}$.

$\{0,1\}^n$ The set of n-bit strings.

$\mathbb{F}_2$ The field with two elements.

$\mathbb{F}_{2^8}$ The AES field, represented as bytes with the AES reduction polynomial.

$\oplus$ Bitwise XOR.

$\wedge$ Bitwise AND.

$\neg$ Bitwise complement.

$\ll$ Left shift.

$\gg$ Right shift.

$\lll$ Rotate left.

$\ggg$ Rotate right.

$\mathsf{HW}(x)$ Hamming weight of x.

x_b Bit b of x, with x_0 the least significant bit.

$[n]$ The set $\{0,\ldots,n-1\}$.

$\leftarrow$ Uniform random sampling.

$\leftarrow_{\$}$ Uniform random sampling from a finite set.

$\Pr[E]$ Probability of event E.

$\mathbb{E}[X]$ Expectation of random variable X.

$\mathsf{negl}(\lambda)$ A negligible function in security parameter λ.

All arithmetic on n-bit words is modulo 2^n unless stated otherwise. Byte order in the trace format is little-endian. The notation $u16_{\mathrm{le}}$ and $u32_{\mathrm{le}}$ denotes little-endian encoding.

3 Formal model

3.1 Values, shares, and reconstruction

Definition 3.1 (Two-share Boolean sharing)**.** For $x \in \{0,1\}^n$, a two-share Boolean sharing is a pair $(s_0, s_1) \in \{0,1\}^n \times \{0,1\}^n$ such that

$$\mathsf{Rec}(s_0, s_1) := s_0 \oplus s_1 = x.$$

The sharing is *fresh* if it is generated as

$$r \leftarrow_{\$} \{0,1\}^n, \qquad s_1 := r, \qquad s_0 := x \oplus r.$$

Definition 3.2 (Masked operation)**.** Let $f : (\{0,1\}^n)^m \to \{0,1\}^n$ be an m-ary operation. A masked implementation f^* takes m fresh sharings and returns a sharing (c_0, c_1) such that

$$\mathsf{Rec}(c_0, c_1) = f(x_1, \ldots, x_m)$$

for all inputs $x_1, \ldots, x_m$, with overwhelming probability over the fresh masks.

Definition 3.3 (Declassification boundary)**.** A declassification boundary is an operation that intentionally reveals a value that was protected by a secret type or a sharing. Examples are `Secret::expose_secret`, `Masked::unmask`, and the ChaCha20 keystream output.

3.2 Leakage functions and first-order game

Definition 3.4 (Leakage function)**.** A leakage function is a map $L : \{0,1\}^m \to \mathcal{Y}$ for some output set $\mathcal{Y}$. A first-order single-share leakage function depends on at most one share of a sharing.

Definition 3.5 (First-order single-share game)**.** Let λ be a security parameter and $n = n(\lambda)$. The game $\mathsf{Exp}^{\mathsf{fo}}_{\mathcal{A},\mathcal{L}}(n)$ proceeds as follows:

1. The challenger samples $x \leftarrow_{\$} \{0,1\}^n$, $r \leftarrow_{\$} \{0,1\}^n$, and sets $s_0 := x \oplus r$, $s_1 := r$.
2. The adversary $\mathcal{A}$ chooses a leakage function $L \in \mathcal{L}$ and a bit $b \leftarrow_{\$} \{0,1\}$.
3. The challenger returns $L(s_b)$ to $\mathcal{A}$.
4. $\mathcal{A}$ outputs a guess b'.

The advantage is

$$\mathsf{Adv}^{\mathsf{fo}}_{\mathcal{A},\mathcal{L}}(n) := \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

Theorem 3.6 (Ideal single-share security)**.** *For any $x \in \{0,1\}^n$, any uniform independent mask r, and any leakage function L that depends on only one share, the distribution of $L(s_b)$ is independent of x. Consequently, for all adversaries $\mathcal{A}$,*

$$\mathsf{Adv}^{\mathsf{fo}}_{\mathcal{A},\mathcal{L}}(n) = 0$$

in the ideal model.

Proof. Each share is uniform on $\{0,1\}^n$ and independent of x: $s_1 = r$ is uniform, and $s_0 = x \oplus r$ is uniform for every fixed x. Therefore the marginal distribution of either share is the uniform distribution, which does not depend on x. Any function of a single share has a distribution that depends only on that share's distribution. □

Remark 3.7 (What the theorem does not say). Theorem 3.6 models an ideal single-share leakage function. It does not model glitches, transitions, coupling between shares, register reuse, or a physical probe that observes a function of both shares. Those effects are outside the current claim and are the reason hardware validation is required.

3.3 Assumptions

Assumption 3.8 (Fresh masks)**.** Every mask is sampled uniformly from $\{0,1\}^n$, is independent of the secret and of all other masks, and is used at most once.

Assumption 3.9 (Entropy)**.** The entropy source is a secure CSPRNG when it reports success. A failure is reported as an error and is never replaced by a weak fallback.

Assumption 3.10 (Constant-time compilation)**.** The compiler preserves the data-independent control flow and memory-access pattern of the constant-time gadgets on the evaluated target.

Assumption 3.11 (Leakage model)**.** The assessment uses a leakage model that is appropriate for the target. A passing test under an inappropriate model is not evidence of absence of leakage.

Assumption 3.12 (No faults or invasive access)**.** The adversary does not inject faults, modify the device, or perform invasive probing.

4 Core data types and invariants

4.1 `Choice`

`Choice` is a one-bit value used for branchless selection. It exposes `from_bool`, `as_u8`, and mask/selection helpers. The invariant is that a `Choice` is represented by a value in $\{0,1\}$ and that no operation on it branches on a secret.

4.2 `Secret<T>`

`Secret<T>` wraps a value and deliberately does not implement `PartialEq`, `Ord`, `Hash`, `Index`, `Clone`, or `Copy`. The only ways to read the value are `expose_secret` and `into_inner`, both of which are declassification boundaries. The type does not guarantee zeroization.

4.3 `Masked<T>`

A `Masked<T>` value is a pair of shares (s_0, s_1). The invariant is

$$s_0 \oplus s_1 = x$$

where x is the logical value. The shares are not exposed except through `shares`, `unmask`, and the explicit analysis boundary. A fresh sharing is constructed by `Masked::mask(value, mask)` with a caller-supplied uniform mask, or by `Masked::from_parts(s_0,s_1)` when the caller has already constructed a valid sharing.

4.4 `EntropySource`

Definition 4.1 (Entropy source contract)**.** An `EntropySource` implementation must either fill the entire destination buffer with cryptographically secure random bytes and return `Ok(())`, or return `Err(EntropyError::Unavailable)` without claiming success. Partial fill followed by success is a contract violation.

4.5 Invariants

Table 1: Core invariants.

ID	Invariant	Enforcement
I1	Reconstruction: $s_0 \oplus s_1 = x$ for every `Masked<T>`.	Tests, Kani
I2	Freshness: every mask is uniform, independent, and used once.	API contract, review
I3	Share independence: each share is uniform under a fresh mask.	Kani, statistical tests
I4	Constant time: no secret-dependent branch or index in the core gadgets.	Assembly, dudect
I5	Fail closed: entropy errors propagate as `Err`.	Tests, code review
I6	No unsafe code in core, reference, platform, or harness.	`forbid(unsafe_code)`
I7	Parser bounds: trace dimensions are checked before allocation.	Tests, fuzzing
I8	Release reproducibility: identical artifact hash across independent builds.	Two-build compare

5 Constant-time primitives

5.1 Comparison

For $n \in \{8, 16, 32, 64, 128\}$, `ct::eq_n(a,b)` returns a `Choice` that is 1 iff $a = b$. The implementation is branchless:

$$\texttt{eq}_n(a, b) = \neg \bigvee_{i=0}^{n-1} (a_i \oplus b_i).$$

The `is_zero_n` family is the same predicate with $b = 0$.

5.2 Selection and swap

For a public `Choice` c,

$$\texttt{select}_n(c, a, b) = (a \wedge \texttt{mask}(c)) \oplus (b \wedge \neg\texttt{mask}(c)),$$

where $\texttt{mask}(c)$ is the all-ones word when $c = 1$ and zero otherwise. The swap operation uses the same mask to exchange two words without a branch.

5.3 Table lookup

`lookup_u8(table, index)` scans all 256 entries and selects the entry whose index matches, without a secret-dependent memory access. The table must have exactly 256 entries. The implementation is constant-time with respect to the index.

Theorem 5.1 (Constant-time lookup correctness)**.** *For every table* $T \in (\{0,1\}^8)^{256}$ *and every index* $i \in \{0,1\}^8$, `lookup_u8`$(T, i) = T[i]$.

Proof. The scan accumulates $T[j]$ exactly when $j = i$, using a branchless equality mask. All other terms are masked out. The result is $T[i]$. □

6 Masked arithmetic

6.1 XOR, complement, shifts, and rotations

These operations are share-wise:

$$(a_0, a_1) \oplus (b_0, b_1) = (a_0 \oplus b_0,\ a_1 \oplus b_1), \tag{1}$$
$$\neg(a_0, a_1) = (\neg a_0,\ a_1), \tag{2}$$
$$(a_0, a_1) \lll k = (a_0 \lll k,\ a_1 \lll k), \tag{3}$$
$$(a_0, a_1) \ggg k = (a_0 \ggg k,\ a_1 \ggg k), \tag{4}$$
$$(a_0, a_1) \ll k = (a_0 \ll k,\ a_1 \ll k), \tag{5}$$
$$(a_0, a_1) \gg k = (a_0 \gg k,\ a_1 \gg k). \tag{6}$$

For shifts, amounts greater than or equal to the word width produce zero. The implementation uses checked shifts and returns zero for out-of-range amounts.

6.2 Refresh

$$\texttt{refresh}((a_0, a_1), r) = (a_0 \oplus r,\ a_1 \oplus r).$$

The logical value is unchanged, and the new shares are randomized by r.

6.3 AND

Algorithm 1 — Masked AND

```
Input:  shares (a0, a1), (b0, b1), fresh uniform mask r
Output: shares (c0, c1)

c0 <- (a0 & b0) ^ r
c1 <- (a0 & b1) ^ (a1 & b0) ^ (a1 & b1) ^ r
return (c0, c1)
```

Theorem 6.1 (Masked AND correctness)**.** *For all* $a, b \in \{0,1\}^n$ *and all valid sharings, the output of Algorithm 1 satisfies* $c_0 \oplus c_1 = a \wedge b$.

Proof. Expand $(a_0 \oplus a_1) \wedge (b_0 \oplus b_1)$ into the four products. The two occurrences of r cancel under XOR. □

Theorem 6.2 (Masked AND first-order freshness)**.** *If* r *is uniform and independent of the inputs, then each output share is uniform and independent of the logical value.*

Proof. c_0 is the XOR of a fixed value with the uniform mask r, hence uniform. c_1 is also the XOR of a fixed value with r, hence uniform. The pair reconstructs $a \wedge b$ by Theorem 6.1. □

6.4 Masked addition

Algorithm 2 — Masked 32-bit addition

```
Input:  shares (a0, a1), (b0, b1), entropy source E
Output: shares (c0, c1)

result <- (0, 0)
carry  <- (0, 0)
for i = 0 .. 31:
    a <- bit(a, i)
    b <- bit(b, i)
    sum <- a XOR b XOR carry
    result <- result XOR sum
    if i == 31: break
    first  <- AND(a, b, E.mask_u32())
    second <- AND(a, carry, E.mask_u32())
    third  <- AND(b, carry, E.mask_u32())
    carry  <- (first XOR second XOR third) << 1
return result
```

Theorem 6.3 (Masked addition correctness)**.** *For all $a, b \in \{0,1\}^{32}$, the output of Algorithm 2 satisfies* $\mathsf{Rec}(c_0, c_1) = (a + b) \bmod 2^{32}$.

Proof sketch. The reconstructed sum bit at position i is $a_i \oplus b_i \oplus c_i$, and the reconstructed carry is $(a_i \wedge b_i) \oplus (a_i \wedge c_i) \oplus (b_i \wedge c_i)$, shifted left by one. This is the standard full-adder recurrence. Induction on i from $c_0 = 0$ gives the 32-bit sum modulo 2^{32}. Each masked AND preserves the reconstructed value by Theorem 6.1, so the masked implementation computes the same function. □

Remark 6.4 (Fresh masks). The correctness proof does not require fresh masks; the security argument does. Reusing a mask across ANDs invalidates Assumption 3.8 and the first-order claim.

6.5 Selection, equality, and bit extraction

Selection selects each share independently with a public `Choice`. Equality compares $a_0 \oplus b_0$ with $a_1 \oplus b_1$ in constant time. Bit extraction masks both shares with $1 \ll i$, returning zero for $i \geq n$. These operations preserve the reconstruction invariant.

7 ChaCha20 specification

7.1 State and constants

The ChaCha20 state is sixteen 32-bit words:

$$\text{state} = (c_0, c_1, c_2, c_3,\ k_0, \ldots, k_7,\ \mathit{counter},\ n_0, n_1, n_2),$$

where the constants are

$$(c_0, c_1, c_2, c_3) = (\texttt{0x61707865}, \texttt{0x3320646e}, \texttt{0x79622d32}, \texttt{0x6b206574}),$$

the key is 256 bits, the counter is 32 bits, and the nonce is 96 bits.

7.2 Quarter round

Algorithm 3 — ChaCha20 quarter round

```
Input:  state words a, b, c, d
Output: updated state words

a <- a + b
d <- (d XOR a) <<< 16
c <- c + d
b <- (b XOR c) <<< 12
a <- a + b
d <- (d XOR a) <<< 8
c <- c + d
b <- (b XOR c) <<< 7
```

7.3 Block function

Algorithm 4 — ChaCha20 block

```
Input:  key (32 bytes), nonce (12 bytes), counter (u32)
Output: 64-byte keystream block

state <- constants || key || counter || nonce
initial <- state
for 10 double rounds:
    quarter_round(state, 0, 4, 8, 12)
    quarter_round(state, 1, 5, 9, 13)
    quarter_round(state, 2, 6, 10, 14)
    quarter_round(state, 3, 7, 11, 15)
    quarter_round(state, 0, 5, 10, 15)
    quarter_round(state, 1, 6, 11, 12)
    quarter_round(state, 2, 7, 8, 13)
    quarter_round(state, 3, 4, 9, 14)
for i = 0 .. 15:
    state[i] <- state[i] + initial[i]
return little_endian_bytes(state)
```

7.4 Masked scalar implementation

The scalar implementation stores the state as sixteen `Masked<u32>` words. The initial state is masked with fresh randomness. For each block, the working state and the initial state used by the final addition are independently refreshed. The quarter round uses masked addition, XOR, and rotation. The final addition uses the masked adder. The keystream output is unmasked only at the declassification boundary.

7.5 Bitsliced batch implementation

The batch implementation processes $B = 64$ blocks. For each of the 16 state words and each of the 32 bit positions, a `Masked<u64>` plane holds one bit from each of the 64 lanes. The quarter round operates on planes. The masked adder is a ripple-carry adder over the 32 bit planes:

$$\mathrm{carry}_{i+1} = (a_i \wedge b_i) \oplus (a_i \wedge \mathrm{carry}_i) \oplus (b_i \wedge \mathrm{carry}_i),$$

with a fresh mask for each AND. After the final addition, the planes are transposed into lane-major output.

Algorithm 5 — Bitsliced masked addition

```
Input:  planes a[0..31], b[0..31], entropy source E
Output: planes result[0..31]

carry <- zero plane
for i = 0 .. 31:
    xor <- a[i] XOR b[i]
    result[i] <- xor XOR carry
    if i == 31: break
    generate   <- AND(a[i], b[i], E.mask_u64())
    propagated <- AND(xor, carry, E.mask_u64())
    carry      <- generate XOR propagated
return result
```

Remark 7.1. Algorithm 5 uses the generate/propagate form of the full adder. It is equivalent to the three-AND form in Algorithm 2 after XOR simplification, but it uses two masked ANDs per bit instead of three.

7.6 Transpose

Algorithm 6 — 64x64 bit-matrix transpose

```
Input:  32 planes, each a u64 row
Output: 64 u32 lane words

rows[0..31] <- unmask(planes[0..31])
for width in [32, 16, 8, 4, 2, 1]:
    mask <- stage_mask(width)
    for row in 0 .. 63:
        if row & width == 0:
            swap <- (rows[row] XOR (rows[row+width] << width)) & mask
            rows[row] <- rows[row] XOR swap
            rows[row+width] <- rows[row+width] XOR (swap >> width)
return rows[0..63] truncated to u32
```

The transpose is a standard butterfly network. It is applied to the unmasked output planes; the unmasking is a declassification boundary at the keystream output. The transpose is validated by differential tests against the scalar and unmasked references.

8 Harness specification

8.1 Trace format

```
TraceFile ::= Magic Version SamplesPerTrace TraceCount Sample*
Magic            ::= "SiCaSTR1"                    (8 bytes)
Version          ::= u16le(1)
SamplesPerTrace  ::= u32le
TraceCount       ::= u32le
Sample           ::= i16le
```

The parser enforces:

1. magic equals `SiCaSTR1`;
2. version equals 1;
3. $1 \leq \text{trace_count} \leq 10^6$;
4. $1 \leq \text{samples_per_trace} \leq 10^6$;

5. trace_count · samples_per_trace $\leq 10^8$;
6. the file contains exactly the declared number of samples.

8.2 Preprocessing

Trim. $T'_{i,j} = T_{i,j+s}$ for $j \in [0, e-s)$.

Filter. $T'_{i,j} = \lfloor \frac{1}{|W_j|} \sum_{k \in W_j} T_{i,k} \rfloor$, where $W_j = [\max(0, j-r), \min(n-1, j+r)]$ and $r = \lfloor w/2 \rfloor$.

Align. Choose $d^* = \arg\max_{d \in [-m,m]} \rho(R, T_i, d)$, then shift T_i by d^* and zero-fill.

8.3 TVLA

Algorithm 7 — Fixed-vs-random TVLA

```
Input:  fixed traces F[0..n0-1], random traces R[0..n1-1], threshold t
Output: report

for sample j = 0 .. samples-1:
    compute means, variances, and Welch t_j
    track max |t_j|, its sample, and the count above t
return report
```

The statistic is

$$t_j = \frac{\bar{F}_j - \bar{R}_j}{\sqrt{s_F^2/n_0 + s_R^2/n_1}},$$

with s^2 the sample variance. The implementation clamps negative variances to zero and returns zero when the denominator is zero.

8.4 CPA

Algorithm 8 — CPA

```
Input:  traces T[0..n-1][0..s-1], plaintexts p[0..n-1], model M
Output: report

for key k = 0 .. 255:
    model[i] <- M(p[i], k)
    for sample j = 0 .. s-1:
        rho[k][j] <- Pearson(model, T[:,j])
    best[k] <- max_j |rho[k][j]|
return argmax_k best[k]
```

Lemma 8.1 (Hamming-weight complement symmetry)**.** *For all $x, k \in \{0,1\}^8$,*

$$\mathsf{HW}(x \oplus k) + \mathsf{HW}(x \oplus k \oplus \texttt{0xFF}) = 8.$$

Therefore the Hamming-weight model has perfectly negatively correlated predictions for k and $k \oplus$ `0xFF`, and the tie-break returns the lower key.

Proof. Complementing all eight bits of a byte maps a byte of weight w to a byte of weight $8 - w$. The sum is 8. □

8.5 DPA

Algorithm 9 — AES S-box DPA

```
Input:  traces T[0..n-1][0..s-1], plaintexts p[0..n-1]
Output: report

for key k = 0 .. 255:
    for bit b = 0 .. 7:
        for sample j = 0 .. s-1:
            S0 <- {i : SBOX[p[i] XOR k]_b = 0}
            S1 <- {i : SBOX[p[i] XOR k]_b = 1}
            if S0 or S1 is empty: continue
            delta <- mean(T[S1][j]) - mean(T[S0][j])
            track max |delta|, signed delta, key, bit, sample
return report
```

Lemma 8.2 (Why the S-box matters)**.** *For a single-bit leakage function $L(p, k) = (p \oplus k)_b$, the absolute difference of means is independent of the key bits other than b. Consequently, a single-bit partition of $p \oplus k$ cannot distinguish the full key in the ideal model.*

Proof sketch. For fixed b, $(p \oplus k)_b = p_b \oplus k_b$. Changing k_b swaps the two partitions; changing any other key bit leaves the partition unchanged. Swapping the partitions negates the difference but preserves its absolute value. Therefore all keys with the same bit b have the same absolute difference, and keys with the other bit value have the same absolute difference as well. The absolute value is independent of the key. □

The AES S-box provides a nonlinear map from $p \oplus k$ to the partition bit, so the partition depends on all key bits. This is why the DPA implementation uses $\mathsf{SBOX}[p \oplus k]_b$ rather than $(p \oplus k)_b$.

8.6 Timing and leakage model

The timing command compares fixed and random classes with Welch's t-test. The leakage-model command computes the Pearson correlation between each share and the secret for masked byte, masked AND, and masked addition. A negative control must be detected. The thresholds and sample counts are parameters and must be published with the result.

9 Security definitions and claims

9.1 First-order security definition

Definition 9.1 (First-order security)**.** An implementation is first-order secure for a leakage class $\mathcal{L}$ if, for every adversary $\mathcal{A}$ and every $L \in \mathcal{L}$, the advantage in Definition 3.5 is negligible in the security parameter.

The ideal argument of Theorem 3.6 gives first-order security for single-share leakage under Assumptions 3.8 and 3.9. The physical implementation is not proven first-order secure; it is designed to satisfy the model and evaluated with local evidence.

9.2 Claims

Table 2: Public claims and status.

ID	Claim	Status
C1	The safe API prevents accidental secret-dependent branching/indexing.	Compile-fail tests pass.
C2	Constant-time gadgets are data-independent on supported targets.	Dudect and assembly evidence on x86_64; other targets pending.
C3	First-order Boolean masking reduces first-order DPA below the TVLA threshold for the reference algorithm.	Synthetic/model evidence; hardware TVLA pending.
C4	Entropy failure is fail-closed.	Local tests pass.
C5	Zero runtime dependencies.	Verified locally.
C6	No unsafe code in core logic crates.	`forbid(unsafe_code)`.
C7	Builds are reproducible.	Two-build compare passes.
C8	Critical invariants are model-checked.	30 Kani harnesses pass.
C9	v1.0 meets SAL-4.	Pending independent verification.
C10	Release artifacts are signed and traceable.	Local signing dry run; production pending.

9.3 Proof obligations

Table 3: Proof obligations and evidence.

ID	Obligation	Evidence / status
PO1	`Choice` and `ct` gadgets are branchless.	Assembly matrix, dudect; pass on x86_64.
PO2	`Secret` prevents accidental equality/indexing/cloning.	Compile-fail doctests; pass.
PO3	`Masked` reconstruction is correct.	Unit tests, Kani; pass.
PO4	Masked AND is correct and fresh.	Kani, tests; pass.
PO5	Masked addition is correct.	Kani, tests; pass.
PO6	Refresh preserves the value and randomizes shares.	Kani, tests; pass.
PO7	Entropy failure propagates.	Tests; pass.
PO8	ChaCha20 matches RFC 8439.	Test vectors; pass.
PO9	Bitsliced batch matches scalar/unmasked.	Differential tests; pass.
PO10	Trace parser rejects malformed input.	Fuzzing, Miri; pass.
PO11	TVLA/CPA/DPA are correct on synthetic data.	Unit tests; pass.
PO12	No unsafe code in core/reference/platform/harness.	Lints, source audit; pass.
PO13	Release artifact is reproducible.	Two-build compare; pass.
PO14	Zeroization drop path erases bytes.	Tests, Miri, assembly; pass for the adapter.
PO15	Performance hard ceiling is met.	Benchmark; 7.15×; pass.
PO16	External audit, red team, hardware, production signing.	Pending.

10 Verification methodology

10.1 Testing

The workspace uses unit tests, property-style randomized tests, differential tests, compile-fail doctests, and integration tests. The core tests cover the constant-time gadgets, secret handling, masking, refresh, reconstruction, and addition. The reference tests cover RFC 8439 vectors, differential checks, lane independence, key-plane balance, batch/scalar interoperability, counter wrap, and entropy failure. The harness tests cover parsing, preprocessing, TVLA, CPA, DPA, timing, synthetic generation, and leakage model checks.

10.2 Kani

Kani (Kani Rust Verifier Project, 2024) is used for bounded model checking of the core gadgets. The 30 harnesses cover constant-time comparison, selection, swap, lookup, masking, refresh, share reconstruction, addition, and share uniformity. The proofs are bounded to the declared integer widths and the assumptions in the harness.

10.3 Miri and sanitizers

Miri (Rust Project, 2024) checks the Rust mid-level IR for undefined behaviour on the covered paths. AddressSanitizer (Serebryany et al., 2012) checks native memory errors. Valgrind (Nethercote and Seward, 2007) checks native memory errors and definite leaks on five test binaries. The statistical, OS, and subprocess paths that Miri cannot execute are documented and run natively.

10.4 Fuzzing and mutation

Five libFuzzer campaigns (LLVM Project, 2024) cover masked operations, constant-time lookup, trace parsing, ChaCha20, and the bitsliced batch path. Mutation testing (Jia and Harman, 2011) measures the strength of the test suite: the current workspace run tests 630 mutants, catches 574, misses 0, and classifies 56 as unviable.

10.5 Assembly and timing

The assembly checker inspects the generated code for secret-dependent branches and memory accesses across optimization levels, codegen units, panic strategies, and LTO variants. The dudect-style timing tests (Reparaz et al., 2017) compare fixed and random classes. The results are local measurements, not a proof for every target.

10.6 Release verification

The release manifest validator checks the artifact hash, checksum file, provenance, SBOM component graph, workspace versions, timestamp, and (optionally) the current commit. The reproducible-build comparison builds the artifact twice in independent target directories and compares the bytes. The local signing dry run signs and verifies a detached signature with an ephemeral test key.

11 Data formats

11.1 Trace and plaintext files

The trace format is defined in Section 8. The plaintext file is a raw byte sequence with one byte per trace. The trace count and plaintext count must match for CPA and DPA.

11.2 Reports

The harness prints human-readable reports. The TVLA report contains `max_abs_t`, `max_sample`, `exceeding_samples`, `sample_count`, `fixed_traces`, `random_traces`, and `threshold`. The CPA report contains `best_key`, `best_correlation`, `best_sample`, and a per-key correlation vector. The DPA report contains `best_key`, `best_bit`, `best_sample`, `best_difference`, and `best_signed_difference`.

11.3 Release manifest

The release manifest directory contains:

`sicas-harness` The release binary.

`SHA256SUMS` The SHA-256 checksum of the binary.

`provenance.json` Project, commit, rustc, host, build command, artifact, SHA-256, and timestamp.

`sbom.cdx.json` CycloneDX SBOM with workspace components and dependency edges.

`README.md` Human-readable summary with commit, host, Rust version, artifact, and SHA-256.

11.4 Evidence bundle

The evidence bundle contains the proof summaries, mutation and coverage evidence, cross-architecture results, release metadata, signing dry-run record, and a source snapshot. The bundle has an `INDEX.md` and a `MANIFEST.sha256`; the manifest is verified with `sha256sum -c MANIFEST.sha256`.

12 Error model and fail-closed behaviour

`EntropyError::Unavailable` The entropy source failed or is not supported. The caller must not fall back to a weak source.

`TraceError` Invalid magic, unsupported version, invalid dimensions, oversized input, length mismatch, or I/O error. The parser returns a typed error and does not panic.

`TvlaError` Empty group, too few traces, dimension mismatch, or invalid threshold.

`CpaError` Empty input, length mismatch, or too few traces.

`DpaError` Empty input, length mismatch, or too few traces.

`PreprocessError` Empty trace, invalid range, invalid window, invalid reference, invalid shift limit, or trace rebuild failure.

The fail-closed principle applies to entropy and to release validation. A missing or invalid input produces an error, not a silent fallback.

13 Performance model

13.1 Complexity

- Scalar ChaCha20: $O(1)$ per 64-byte block; the state is fixed-size.
- Bitsliced batch: $O(B)$ work for $B = 64$ blocks, with $O(1)$ state per lane and a 64×64 transpose.
- TVLA: $O(N \cdot S)$ time and $O(S)$ auxiliary space.
- CPA: $O(256 \cdot N \cdot S)$ time and $O(S)$ auxiliary space per key.
- DPA: $O(256 \cdot 8 \cdot N \cdot S)$ time and $O(S)$ auxiliary space.

13.2 Measured results

The final full proof measures a scalar masked overhead of approximately 320× and a bitsliced batch overhead of 7.15× versus the unmasked reference. The 10× hard ceiling is met. The 5× target is open; reaching it requires SIMD or a faster audited mask stream.

14 Conformance and test vectors

14.1 RFC 8439

The reference tests include the RFC 8439 ChaCha20 block test vector and the encryption test vector. The masked scalar and bitsliced paths are checked against the unmasked reference for randomized keys, nonces, and counters. The tests also cover counter wrap and batch/scalar interoperability.

14.2 Differential statistics

The TVLA, CPA, and DPA statistics are validated against an independent Python model over 200 deterministic cases. The leakage-model check includes a negative control that must be detected.

14.3 Release conformance

The release manifest validator checks that the artifact hash, checksum file, provenance, SBOM, workspace versions, timestamp, and commit agree. The reproducible build comparison checks that two independent builds produce identical bytes.

15 Versioning and change log

The document and the code are versioned separately. The document version follows semantic versioning: major changes to the specification, minor additions, and patch clarifications. The code version is the workspace version `0.1.0` until the external gates pass. Every proof run records the code commit and the evidence tag.

Table 4: Yellow paper revision history.

Version	Date	Code / evidence	Changes
0.1.0	2026-10-30	initial tree	Core types, masking model, trace format.

Version	Date	Code / evidence	Changes
0.9.0	2026-10-31	`8c41e4b`	Full proof obligations, evaluation mapping, release format.
1.0.0	2026-10-31	`ef13675`, `final-audit-2026-10-31-dpa`	AES S-box DPA, GPL-3.0-only, final audit package.

A API index

Table 5: Public API index (selected).

Item	Role
`Choice::from_bool`, `as_u8`	Branchless one-bit value.
`ct::eq_*`, `select_*`, `swap_u32`, `lookup_u8`	Constant-time comparison, selection, swap, and lookup.
`Secret::new`, `expose_secret`, `into_inner`	Opaque secret wrapper and declassification.
`Masked::mask`, `from_parts`, `shares`, `unmask`	Sharing construction and reconstruction.
`Masked::refresh`, `xor`, `and`, `add_masked`	Masked operations.
`EntropySource::fill_bytes`, `mask_u32`, `mask_u64`	Fail-closed entropy contract.
`MaskedChaCha20::new`, `keystream_block`, `keystream_blocks`	Masked ChaCha20 scalar and batch.
`TraceSet::from_traces`, `read_from`, `write_to`	Trace storage and I/O.
`tvla`, `cpa_hamming_weight`, `cpa_with_model`, `dpa_bit`	Leakage analysis.
`preprocess::trim`, `moving_average`, `align`	Trace preprocessing.
`timing::dudect`, `leakage::check_all`	Timing and leakage-model checks.
`OsEntropy::new`	Safe OS entropy adapter.
`zeroize_bytes`, `ZeroizingBytes`	Experimental zeroization boundary.

B Conformance checklist

1. Build the workspace with the pinned toolchain.
2. Run all workspace tests and doctests.
3. Run the quick local proof.
4. Run the full local proof for the release candidate.
5. Verify the evidence bundle manifest.
6. Verify the release manifest and artifact hash.
7. Compare the release SHA-256 with the published value.
8. Check the claim-to-evidence map for the claimed assurance level.
9. Record every external gate that remains open.

References

Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011. doi: 10.1109/TSE.2010.62.

Kani Rust Verifier Project. Kani rust verifier. https://github.com/model-checking/kani, 2024. Accessed 2026-10-31.

LLVM Project. libFuzzer: A library for coverage-guided fuzz testing. https://llvm.org/docs/LibFuzzer.html, 2024. Accessed 2026-10-31.

Nicholas Nethercote and Julian Seward. Valgrind: A framework for heavyweight dynamic binary instrumentation. In *Proceedings of the ACM SIGPLAN 2007 Conference on Programming Language Design and Implementation (PLDI 2007)*, pages 89–100, 2007. doi: 10.1145/1250734.1250746.

Oscar Reparaz, Josep Balasch, and Ingrid Verbauwhede. Dude, is my code constant time? In *Design, Automation & Test in Europe Conference (DATE 2017)*, pages 1697–1702, 2017. doi: 10.23919/DATE.2017.7927267.

Rust Project. Miri: An interpreter for Rust's mid-level intermediate representation. https://github.com/rust-lang/miri, 2024. Accessed 2026-10-31.

Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitry Vyukov. AddressSanitizer: A fast address sanity checker. In *Proceedings of the 2012 USENIX Annual Technical Conference (USENIX ATC 2012)*, pages 309–318, 2012.